

Resumen Conceptos y Paradigmas de Lenguajes de Programación

Clase 1

Introducción y evaluación de lenguajes

Introducción

Un lenguaje de programación es un lenguaje formal diseñado para expresar algoritmos y describir procesos computacionales que pueden ser ejecutados por una máquina.

Proporciona un conjunto de reglas sintácticas y semánticas que permiten especificar instrucciones que serán interpretadas o compiladas en operaciones ejecutables por un sistema informático.

Puede seguir o respetar uno o varios paradigmas de programación, que guían y dictan cómo se escriben y conceptualizan los programas.

Un paradigma de programación indica una forma de pensar la programación. Define un modelo, métodos de realizar cálculos y la manera en que se deben estructurar y organizar las tareas que debe llevar a cabo un programa.

Ejemplos:

Se tiene el paradigma imperativo como la programación estructurada, la orientada a objetos; el paradigma declarativo que incluye la programación funcional, la programación lógica.

Principales componentes

Sintaxis

Es el conjunto de reglas sintácticas y gramaticales que determinan cómo se deben estructurar las instrucciones del lenguaje.

Define la forma válida de los programas

Suele describirse mediante gramáticas formales (por ejemplo, BNF o EBNF)

Semántica

Define el **significado** de las construcciones sintácticas

- Semántica operacional: Describe cómo se ejecuta un programa paso a paso.
- Semántica denotacional: Asocia construcciones del lenguaje con objetos matemáticos.
- Semántica axiomática: Permite razonar formalmente sobre la corrección de programas.

Sistema de tipos

Establece reglas sobre los tipos de datos y su uso.

Funciones principales:

1. Detectar errores
2. Garantizar consistencia
3. Mejorar seguridad del programa

Ejemplos:

- tipado estático vs. dinámico
- tipado fuerte vs. débil

Modelo de ejecución

Define cómo se ejecuta un programa.

Compilado: traducido a código máquina antes de ejecutarse

Interpretado: ejecutado por un intérprete y traducido en tiempo de ejecución

Híbrido: bytecode + máquina virtual

Mecanismos de abstracción

Proporcionan **abstracciones computacionales y de complejidad**

Ejemplos:

- Estructuras de datos
- Estructuras de control

- Módulos y funciones
- Clases y objetos
- Concurrencia

Estudio de conceptos de lenguajes de programación

- Aumentar la capacidad para producir software.
- Mejorar el uso del lenguaje
- Elegir mejor un lenguaje
- Facilitar el aprendizaje de nuevos lenguajes
- Facilitar el diseño e implementación de lenguajes

Criterios de evaluación

Criterios para analizar la calidad del diseño de un lenguaje de programación:

- Simplicidad y legibilidad
- Claridad en los bindings
- Confiabilidad
- Soporte
- Abstracción
- Ortogonalidad
- Eficiencia

Simplicidad y Legibilidad

Los lenguajes simples y legibles:

- producen programas fáciles de escribir y de leer.
- fáciles a la hora de aprenderlo o enseñarlo
- con sintaxis clara y pocas reglas complejas
- evitan ambigüedades

Permiten:

- En escritura: expresividad, rapidez al programar.
- En legibilidad: favorece el mantenimiento del software, trabajo en equipo y revisión de código.

Ejemplo de cuestiones que atentan contra esto:

- Muchas componentes elementales, operadores sobrecargados
- El mismo concepto semántico -- distinta sintaxis
- Distintos conceptos semánticos -- misma notación sintáctica

Claridad en los bindings

Es la asociación entre un nombre y una entidad del programa.

Ejemplos:

- nombre de variable y su valor
- nombre de función y su implementación

Claridad en los bindings implica:

- saber cuándo y hasta cuando ocurre la asociación
- reglas de scope (alcance) claras y no ambiguas.

Momentos posibles:

- Definición del lenguaje
- Implementación del lenguaje
- En escritura del programa
- Compilación
- Cargado del programa
- En ejecución

Tipos de binding:

- Estático(en compilación)
- Dinámico (ejecución)

Confiabilidad

Está relacionada con la seguridad.

Un lenguaje es confiable cuando:

- Evita errores comunes
- Ofrece tipado fuerte
- Detecta errores en compilación

- Previene comportamientos inesperados
Implica:
- Chequeo de tipos: Cuanto antes se encuentren errores, menos costo resulta realizar los arreglos que se requieran.
- Manejo de excepciones: La habilidad para interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.

Soporte

Un lenguaje con buen soporte facilita:

- Aprendizaje
 - Desarrollo
 - Mantenimiento
- El soporte puede consistir en:
- Compiladores e interpretes
 - Bibliotecas
 - Frameworks
 - Herramientas de depuración

Abstracción

Permite en programación:

- Representar conceptos complejos mediante estructuras más simples.
 - Definir y usar estructuras u operaciones complicadas de manera que sea posible ignorar muchos detalles.
 - Ocultar detalles de implementación.
 - Reutilizar código.
 - Manejar complejidad
- Ejemplos:
- Abstracción de datos: tipo de datos abstractos, clases
 - Abstracción en procesos: funciones, módulos, interfaces.

Ortogonalidad

Significa que las características del lenguaje funcionen de manera independiente y consistente entre sí.

Significa en programación:

- Que un conjunto pequeño de constructores primitivos puedan combinarse sin restricciones mediante un conjunto consistente y uniforme de reglas de combinación.
- Cada combinación es legal y con sentido.
La ortogonalidad reduce:
- Excepciones en las reglas
- Complejidad del lenguaje

Eficiencia

La eficiencia hace referencia al uso eficiente de los recursos computacionales por parte del programa generado por el lenguaje de programación.

Sintaxis

Sintaxis y semántica

Un lenguaje de programación ofrece una notación formal para describir algoritmos a ser ejecutados en una computadora.

Requiere de una sintaxis - si es válido el programa - y semántica - qué significa - tanto para la programación como para su traducción.

Sintaxis: conjunto de reglas que definen cómo componer letras, dígitos y otros caracteres para formar los programas.

Semántica: conjunto de reglas para dar significado a los programas sintácticamente válidos.

Permite escribir programas correctos y válidos sintácticamente. Establece reglas para combinar componentes básicas, llamadas **word** y formar sentencias y programas.

Elementos que la componen:

- alfabeto o conjunto de caracteres
- identificadores
- operadores

- palabra clave y palabra reservada
- comentarios y uso de blancos

Contempla cuestiones:

- legibilidad
 - verificabilidad
 - traducción
 - Falta de ambigüedad
-

Sintaxis

Alfabeto:

- Con qué conjunto de caracteres se trabaja
- La secuencia de bits que compone cada caracter determina la implementación.

Identificadores

- Por ejemplo conformado por letras y dígitos, que deben comenzar con una letra.
- Si se restringe la longitud se pierde legibilidad.

Operadores

- Uniformes: con los operadores de suma, resta, etc. La mayoría de los lenguajes utilizan +,-.
- Poco uniformes: otros operadores no hay tanta uniformidad (**|^).

Comentarios

- Para hacer los programas más legibles.

Palabra clave y palabra reservada

- **Palabra clave o keywords:** son palabras claves que tienen un significado dentro de un contexto.
- **Palabra reservada:** Son palabras que además no pueden ser usadas por el programador como identificador de otra entidad.

Ventajas de su uso: Hacen los programas más legibles y permiten una rápida traducción.

Ejemplos de lenguajes con uso de palabras reservadas:

- C ej.:auto, break, case, char, const, continue, default, do ,double, else, enum, extern, float, for, goto, if, int, etc

Estructura sintáctica

Vocabulario o words

- Conjunto de caracteres y palabras necesarias para construir expresiones, sentencias y programas. Ej: identificadores, operadores, palabras claves, etc.
- Las words no son elementales, se construyen a partir del alfabeto.

Expresiones

- Son funciones que a partir de un conjunto de datos devuelven un resultado.
- Son bloques sintácticos básicos a partir de los cuales se construyen las sentencias y programas.

Sentencias

- Componente sintáctico más importante.
- Tiene un fuerte impacto en la facilidad de escritura y legibilidad.

Reglas léxicas y sintácticas

Reglas léxicas

- Conjunto de reglas para formar las "word", a partir de los caracteres del alfabeto.
- Por ej. diferencias entre mayúsculas y minúsculas, símbolo distinto != o <>.

Reglas sintácticas

- Conjunto de reglas que definen cómo son las **sentencias** y **expresiones**

Tipo de sintaxis

- Abstracta
 - Se refiere básicamente a la estructura
- Concreta
 - Se refiere a la parte léxica
- Pragmática
 - Uso práctico

Definición de la sintaxis

Se necesita una descripción finita para definir un conjunto infinito (conjunto de todos los programas bien escritos).

Formas de definir la sintaxis:

- Lenguaje natural: Ej. Fortran
- Gramáticas libres de contextos:
 - Backus y Naun Form: BNF Ej: Algol
 - EBNF
 - Diagramas sintácticos equivalentes a BNF pero más intuitivos

BNF (Backus Naun Form)

- Es una notación formal para describir la sintaxis
- Es un metalenguaje
- Utiliza metasímbolos
 - <, >, ::=, |
- Define las reglas por medio de "producciones"

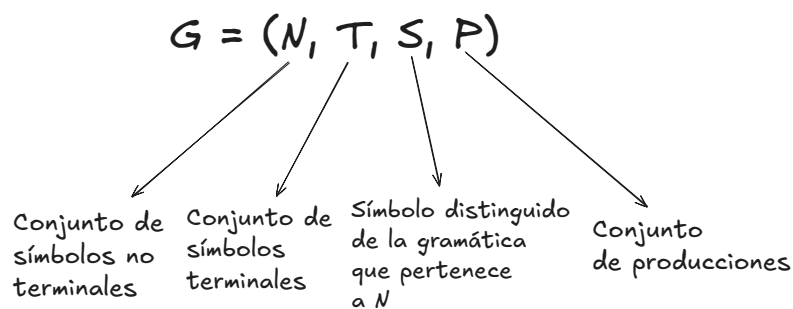
Ejemplo:

No terminal	Metasímbolo	Se define como Terminales
<digito>	::=	0 1 2 3 4 5 6 7 8 9

Gramática

Es un conjunto de reglas finita que define un conjunto infinito de posibles sentencias válidas en el lenguaje.

Una gramática esta formada por una 4-tupla



Parsing y Árboles sintácticos

- Es una oración sintácticamente incorrecta
- No todas las oraciones que se pueden armar con los terminales son válidas
- Se necesita de un Método de análisis (reconocimiento) que permita determinar si un string dado es válido o no en el lenguaje: **Parsing**.

El parse, para cada sentencia construye un "**árbol sintáctico o árbol de derivación**"

Árboles sintácticos

Dos maneras de construirlo:

- Método bottom-up
 - De izquierda a derecha
 - De derecha a izquierda
- Método top-down
 - De izquierda a derecha
 - De derecha a izquierda

Producciones recursivas

- Son las que hacen que el conjunto de sentencias escrito sea infinito.
- Ejemplo:

$$\langle \text{natural} \rangle ::= \langle \text{digito} \rangle \mid \langle \text{digito} \rangle \langle \text{digito} \rangle \mid \dots \mid \langle \text{digito} \rangle \dots \langle \text{digito} \rangle$$

- Si lo planteamos recursivamente:

$GN = (N, T, S, P)$

$N = \{ \text{<natural>, <digito> } \}$ $T = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$

$S = \text{<natural>}$

$P = \{ \text{<natural>} ::= \text{<digito>} \mid \text{<digito> <natural>,} \\ \text{<digito>} ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \}$

Important

Cualquier gramática que tiene una producción recursiva describe un lenguaje infinito.

- Regla recursiva por la izquierda
 - La asociatividad es por la izquierda
 - El símbolo no terminal de la parte izquierda de una regla de producción, aparece al comienzo de la parte derecha
- Regla recursiva por la derecha
 - La asociatividad es por la derecha
 - El símbolo no terminal de la parte izquierda es una regla de producción, aparece al final de la parte derecha

Gramáticas ambiguas

Una gramática es ambigua si una sentencia puede derivarse de más de una forma.

Important

La filosofía de composición es la forma en que trabajan los compiladores

Gramáticas libres de contexto y sensibles al contexto

int y:

y= 22.75

- La gramática libre de contexto no puede verificar:
 - Si x fue declarada antes

- Si los tipos son compatibles
- No considera lo que se define antes ni después.
- Según nuestra gramática son sentencias sintácticamente válidas, aunque puede suceder que a veces no lo sea semánticamente.

Gramática libre de contexto (CFG)

- Es aquella en la que no realiza un análisis de contexto.
- Describen las sintaxis de los lenguajes como C, Java, Python

Gramática sensible al contexto (CSG)

- Se realizan verificaciones que consideran el contexto del programa (Algol 68)
- Se manejan en la fase del análisis semántico del compilador, para detectar ciertas restricciones semánticas.
- Se pueden utilizar en IA en aplicaciones de machine learning, reconocimiento de voz, en lingüística para especificar reglas para lenguaje natural.

EBNF BNF extendido

- EBNF es una **gramática libre de contexto**
- Simplifica la escritura
- Elimina la recursión y la reemplaza por reiteración

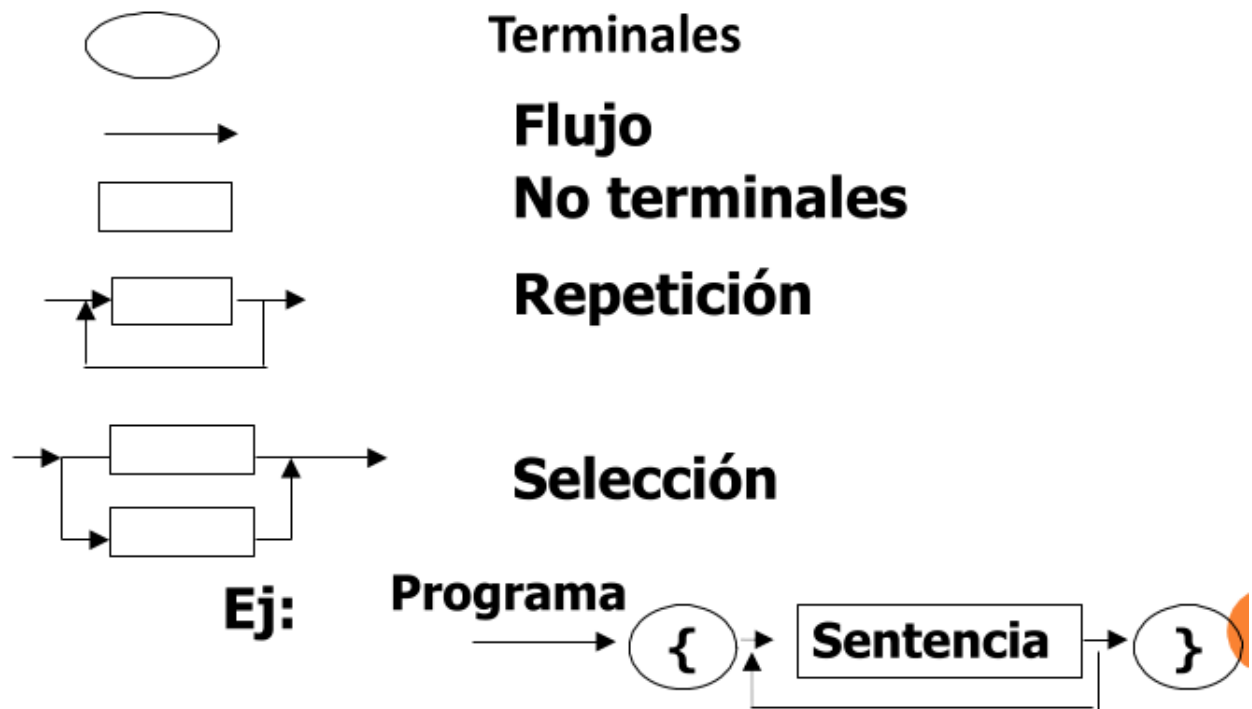
Meta símbolos incorporados:

- $[]$ elemento optativo puede o no estar
- $()$ selección de una alternativa
- $\{ \}$ repetición
- 0 o más veces + 1 o más veces

Diagramas sintácticos (Conway):

- Es un grafo sintáctico o carta sintáctica

- Cada diagrama tiene una entrada y una salida, y el camino determina el análisis.
- Cada diagrama representa una regla o producción
- Para que una sentencia sea válida, debe haber un camino desde la entrada hasta la salida que la describa.
- Se visualiza y entiende mejor que BNF o EBNF



Clase 2

Semántica

- La semántica describe el significado de los símbolos, palabras y frases de un lenguaje ya sea lenguaje natural o lenguaje informático
- La semántica describe el significado de programas sintácticamente válidos/correctos para un lenguaje determinado
- Así se da significado a las construcciones del lenguaje
- La semántica no es igual en todos los lenguajes.
- Código similar en distintos lenguajes puede dar distintas soluciones o reportar errores.
- Una cosa es formato escrito y otra que significa y como se comporta

La semántica no es igual en todos los lenguajes

Semántica - Continuación

- Hay que detectar otros errores semánticos
- Hay características de la estructura de los lenguajes de programación que son difíciles o imposibles de describir con las gramáticas BNF/EBNF.
 - En Java, un valor de punto flotante no se puede asignar a una variable de tipo entero, la inversa está permitido. (Difícil en BNF/EBNF, requiere gramática extensa)
- Se debe usar otro tipo de gramática
- Hay que ver que errores se pueden detectar antes de ejecutar y durante ejecución

Semántica estática (previo a la ejecución)

- Se la llama así porque el análisis para el chequeo se hace en compilación (antes de la ejecución).
- No está relacionada con el significado de la ejecución del programa, está más relacionado con las formas válidas (con la sintaxis).
- El análisis está ubicado entre el análisis sintáctico y el análisis de semántica dinámica, pero más cercano a la sintaxis.
- **Lo que BNF/EBNF no puede ver:** Una gramática puede decir que la estructura `variable = valor` es válida sintácticamente, pero no puede saber si el `valor` es compatible con el tipo de la `variable` o si esa variable fue declarada previamente (ya que son libres de contexto).
- **Ejemplos de errores detectados:**
 - **Compatibilidad de tipos:** Intentar asignar un texto a un entero (`int x = "hola"`). La estructura es correcta, pero el contenido es erróneo para el lenguaje.
 - **Variables duplicadas o no declaradas:** Verificar que cada identificador tenga una única definición y que exista antes de ser usado.

Gramática de atributos

- Para describir la sintaxis y la semántica estática formalmente sirven las denominadas gramáticas de atributos, inventadas por Knuth en 1968.

- Son una extensión de las Gramáticas Libres al Contexto que agregan información adicional para dar/relacionar con el significado.
- Son un enfoque formal tanto para describir como para comprobar la corrección de las reglas semánticas estáticas de un programa
- La usan los compiladores, antes de la ejecución
- Generalmente resuelven los aspectos de la semántica estática.

Funcionamiento

- A las construcciones del lenguaje asociados a símbolos de la gramática (símbolos terminales o no terminales) se les asocia información a través de atributos, que sirven para detectar errores
- Un atributo puede ser:
 - valor de una variable,
 - tipo de una variable o expresión,
 - lugar que ocupa una variable en memoria,
 - dígitos significativos de un número,
 - etc.
- Los valores de los atributos se obtienen mediante las llamadas “ecuaciones o reglas semánticas” asociadas a las producciones gramaticales.
- Las reglas gramaticales son similares a BNF
- Las reglas semánticas (ecuaciones) permiten detectar errores y obtener valores de atributos.
- Los atributos están directamente relacionados a los símbolos gramaticales (terminales y no terminales)
- Las GA se suelen expresar en forma tabular para obtener el valor del atributo

Ejemplo:

decl int X,Y

Regla gramatical	Reglas semánticas (obtener el valor)
Regla 1	Ecuaciones de atributo asociadas
....	
Regla N	Ecuaciones de atributo asociadas

- Usa la tabla y busca si encuentra la producción/regla, si la encuentra de la otra columna usa la ecuación que me permite llegar a los atributos.
- Mira las Reglas (símil BNF/EBNF) y busca atributos para terminales y no terminales.
- Si encuentra el atributo debe llegar a obtener su valor.
- Para obtenerlo genera ecuaciones.
- Para una declaración como:
 - `int x,y;`
 - Se reconoce el tipo (`int`)
 - Se propaga a la lista de variables
 - Se asigna a cada identificador (`x`, `y`)
 - Se guardan en la tabla de símbolos

Resultado:

Variable	Tipo
x	int
y	int

Proceso del compilador

- Análisis léxico (Scanner) → salen lexemas (palabras) y tokens (palabras clave, identificadores, operadores)
- Análisis sintáctico (Parser) → toma los tokens y reconoce la estructura gramatical, y se construye el árbol sintáctico
- Análisis semántico → usa gramática de atributos, se calculan los atributos, y se actualiza tabla de símbolos. Se construye el árbol sintáctico atribuido (se agregan los atributos calculados al árbol sintáctico)
- Siguen fases posteriores del compilador....
- Arbol sintáctico atribuido
 - Parte del Árbol sintáctico y de la gramática de atributos para ver si cumple y detectar errores
 - Esto permite la detección de errores cuando se agregan símbolos a la tabla de símbolos.
 - Si hay errores no pasa a ejecución

Gramática de atributos - Final

Cuando se ejecutan las reglas semánticas (las ecuaciones de la gramática de atributos), pasan varias cosas importantes en el compilador.

Al ejecutar las reglas semánticas se logra:

- Guardar variables en la tabla de símbolos (nombre, tipo, etc.)
 - Detectar errores antes de la ejecución
 - Identificar variables duplicadas
 - Verificar compatibilidad de tipos
 - Controlar reglas propias del lenguaje
 - Detectar usos no permitidos
 - Generar información para las siguientes etapas del compilador
- Detectar todos los errores posibles antes de ejecutar el programa que no pueden ser detectados solo por la sintaxis.

Semántica dinámica

Describe el significado de ejecutar las construcciones del lenguaje; su efecto se ve durante la ejecución. Los programas solo se ejecutan si son correctos en sintaxis y semántica estática. Es difícil de definir formalmente (no existen herramientas simples como BNF) y difícil de modelar ante distintas plataformas.

Diferentes enfoques más utilizados

Formales - complejas:

- Semántica axiomática
 - Semántica denotacional
- La usan los diseñadores de compiladores, para especificar o razonar sobre el comportamiento de los programas.

No formal:

- Semántica operacional
- Por ejemplo se usan para manuales de lenguajes
- En todos se definen Reglas para comprobar la ejecución, la exactitud de un lenguaje, comparar funcionalidades de distintos programas,

especificar el significado.

- Se pueden usar combinados
- No todos sirven para todos los tipos de lenguajes de programación!

Cuadro a modo de resumen

Enfoque de semántica	Idea principal	Cómo describe el programa	Uso principal
Semántica operacional	Describe la ejecución paso a paso	Mediante estados de una máquina abstracta y transiciones	Especificar cómo se ejecuta un programa
Semántica denotacional	Describe el significado matemático	Asocia cada construcción del lenguaje con una función matemática	Modelos formales de lenguajes
Semántica axiomática	Describe propiedades lógicas del programa	Usa axiomas y reglas de inferencia	Verificación de programas

Procesamiento de los programas

Traducción

- Las computadoras ejecutan un lenguaje de bajo nivel llamado “lenguaje de máquina” (con 0’s y 1’s)
- Hoy los programas están escritos en lenguajes de más alto nivel

Interpretación a compilación

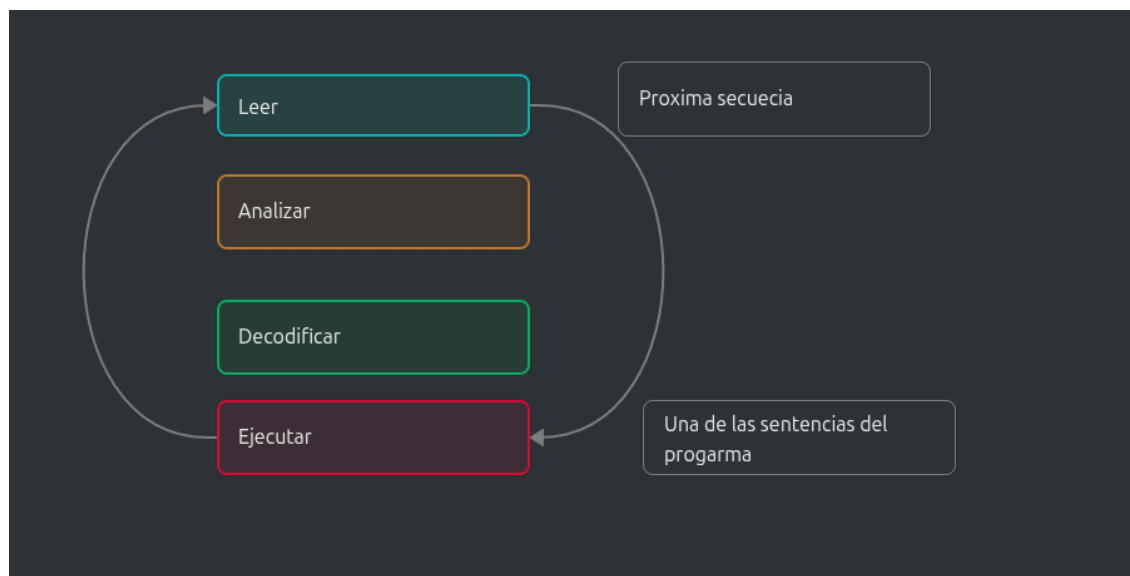
¿Cómo los programas escritos en un lenguaje de alto nivel pueden ser ejecutados sobre una computadora cuyo lenguaje es muy diferente y de muy bajo nivel que entiende 0s y 1s?

- Con Programas Procesadores del lenguaje
- Alternativas:
 - Interpretación

- Compilación
- Interpretación y Compilación (combinación)

Interpretación

- Hay un programa escrito en lenguaje de programación interpretado
- Hay un programa llamado intérprete que realiza el procesamiento de ese lenguaje interpretado en el momento de ejecución
- Ej: Lisp, Smalltalk, Basic, Python, Ruby, PHP, Perl, etc.
- El proceso que realiza cuando se ejecuta sobre cada una de las sentencias del programa es:



- Sólo pasa por ciertas instrucciones, no por todas, según sea la ejecución (ventajas y desventajas)
- Cada vez que vuelvo a ejecutar el programa se repite toda la secuencia.

El intérprete suele contar con una serie de herramientas para realizar el procesamiento/traducción a lenguaje de máquina

- Por cada constructor del lenguaje hay un subprograma en lenguaje de máquina que ejecuta esa acción
- La interpretación se realiza llamando a estos **subprogramas** en la secuencia adecuada hasta generar el resultado de la ejecución

Compilación

- Hay un programa escrito en lenguaje de alto nivel

- Hay un programa llamado **Compilador** que realiza la traducción a lenguaje de máquina
- Ejemplos: Ada, C, C++, Fortran, Pascal, etc.
- El compilador, antes de la ejecución, toma todo el programa en un lenguaje de alto nivel (lenguaje fuente) y lo traduce a un equivalente a lenguaje de máquina.
- Pasa por todas las instrucciones antes de la ejecución (ventajas y desventajas)
- El código que se genera se guarda y se puede reusar ya compilado.
- La compilación implica varias etapas, que analizaremos luego.....



- - Se puede generar:
 - Código objeto generalmente el ejecutable en lenguaje de máquina
 - Código de nivel intermedio en lenguaje ensamblador

Comparación - Compilador e Intérprete

Característica	Compilador	Intérprete
Momento	Antes de la ejecución.	Durante la ejecución.
Orden	Físico (recorre todo el código).	Lógico (solo lo que se ejecuta).
Traducción	Todo de una vez (genera código objeto).	Línea por línea (decodifica y ejecuta).
Eficiencia	Alta: El hardware lo corre directo.	Baja: Repite el proceso en cada lazo.
Memoria	Menor: Solo queda el ejecutable.	Mayor: Requiere intérprete + fuente.

Característica	Compilador	Intérprete
Errores	Difícil de ubicar (pierde referencia).	Fácil de ubicar y corregir.
Privacidad	El código fuente es privado.	El código fuente es público.

Aspecto	Interpretación	Compilación
Momento de traducción	Durante la ejecución	Antes de ejecutar el programa
Unidad de ejecución	Sentencia por sentencia	Programa completo
Velocidad de ejecución	Más lenta	Más rápida
Detección de errores	Aparecen cuando se ejecuta la sentencia	Muchos errores se detectan antes de ejecutar
Depuración	Más fácil (ejecución paso a paso)	Puede requerir recompilación
Uso de memoria	Intérprete + programa + datos	Ejecutable compilado + datos
Portabilidad	Alta si existe intérprete para la plataforma	Requiere recompilar para cada arquitectura
Distribución del programa	Se distribuye el código fuente o script	Se distribuye un ejecutable

62

Técnicas utilizadas

1. Interpretación pura
2. Compilación pura
3. Traducción híbrida

Combinación de Técnicas de Traducción:

- Interpretación y Compilación
- Compilación y Interpretación

En ciertos entornos de programación se combinan para sacar provecho de sus diferencias, reporte de errores, etc.

Interpretación y compilación

El intérprete se utiliza en la etapa de desarrollo para facilitar el diagnóstico y depuración de errores.

- Se compila el programa validado para generar un código objeto más eficiente.

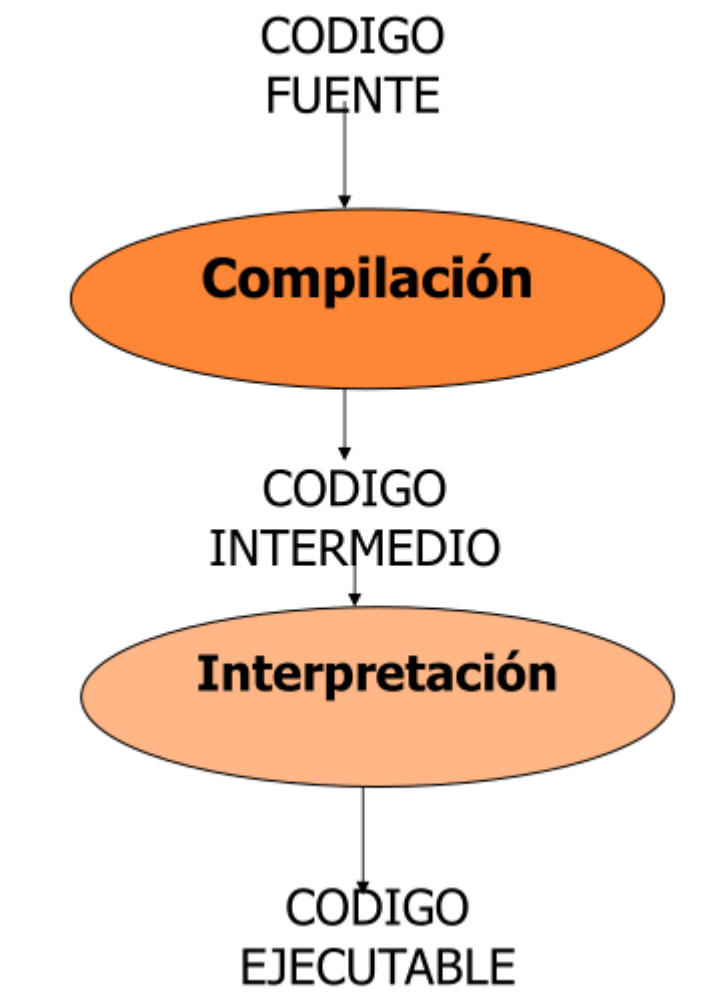
Ej: QuickBasic, Lisp, etc.

Compilación a código intermedio + ejecución en MV

- Se realiza traducción inicial a un código intermedio independiente de la máquina.
Ese código luego se ejecuta en una máquina virtual o sistema de ejecución.
- Esto permite generar programas portables, es decir, que pueden ejecutarse en diferentes máquinas y arquitecturas.

Ejemplos:

- Java (compilador Java genera bytecode. Este es ejecutado por Java Virtual Machine (JVM) en la máquina destino)
- C# y VB.NET (Visual Basic)



Compiladores

¿Cómo funcionan?

- Traducen todo el programa
- Pueden generar un "código ejecutable"(.exe)
- Pueden generar un "código intermedio"(.obj)
- La compilación puede ser en 1 o 2 etapas
Ej: Ada, Pascal, C.
- Estructura de un compilador. Etapas:
 - 1.Etapa de Análisis:
 - Análisis léxico (Scanner)
 - Análisis sintáctico (Parser)
 - Análisis semántico (Semántica estática)
Trabaja sobre el programa fuente. Depende del lenguaje
 - 2.Etapa de Síntesis
 - Optimización del código
 - Generación del código final
Puede generarse: código intermedio y optimización
Genera el programa objeto, Depende de la máquina

Etapas del Compilador (Resumen Ejecutivo)

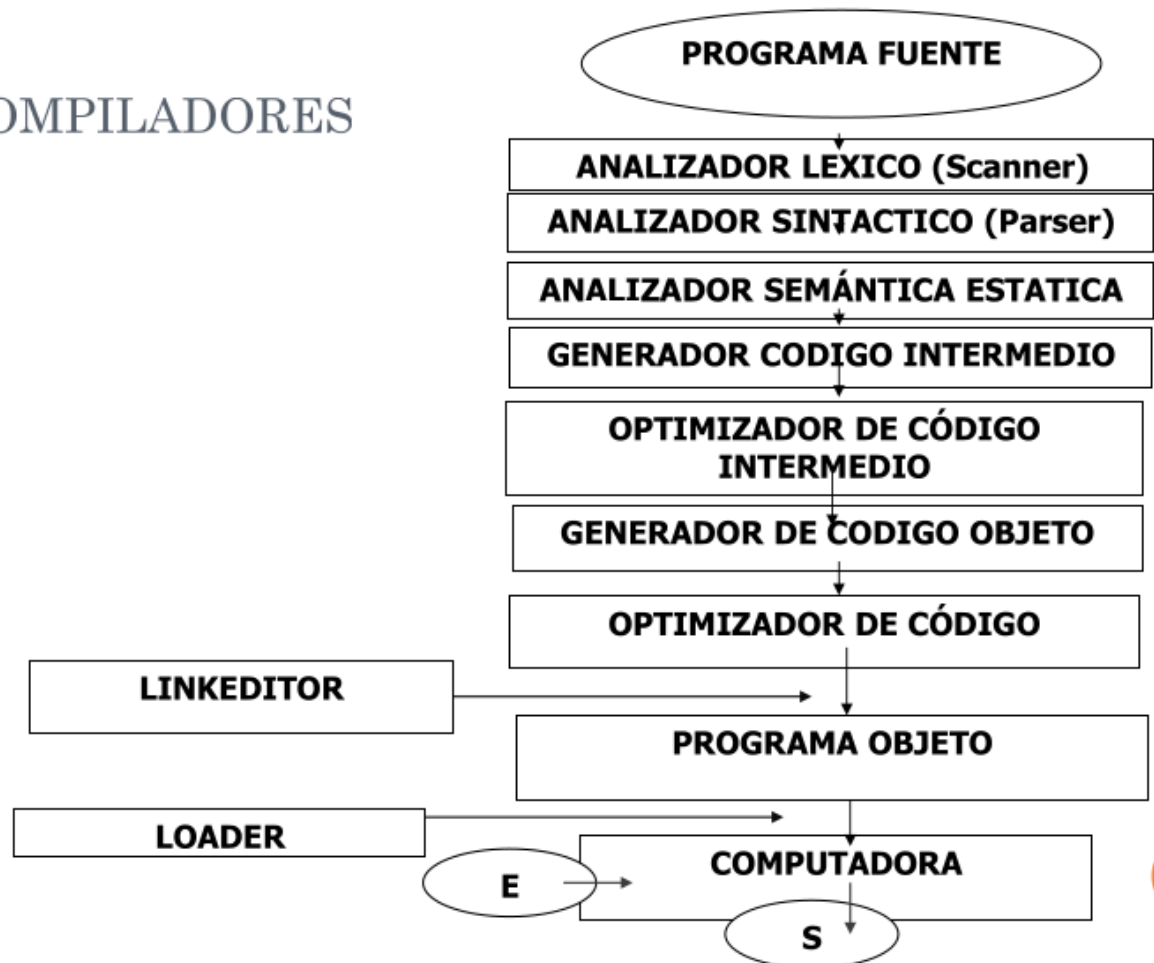
Etapa de Análisis (Depende del Lenguaje)

- **Análisis Léxico (Scanner):** Lee el texto fuente y lo agrupa en **Tokens** (palabras clave, identificadores). Filtra lo innecesario (comentarios, espacios) e inicia la **Tabla de Símbolos**.
- **Análisis Sintáctico (Parser):** Toma los tokens y verifica que sigan las reglas gramaticales. Su salida es el **Árbol Sintáctico** (la estructura jerárquica).
- **Análisis Semántico:** Es el "puente". Revisa que el árbol tenga sentido: **compatibilidad de tipos** y que las variables existan antes de usarse. Genera el **Árbol Atribuido**.

Etapa de Síntesis (Depende de la Máquina)

- **Generación de Código Intermedio:** Traduce el programa a una representación para una **máquina abstracta** (ej. código de tres direcciones). Es independiente del hardware.
- **Optimización (Opcional):** Mejora el código para que sea más rápido o ocupe menos memoria (ej. eliminando código que nunca se ejecuta).
- **Generación de Código Final:** Traduce todo a instrucciones reales del **procesador (Código Objeto)**. Aquí actúan el *Linker* (une librerías) y el *Loader* (carga en memoria).

COMPILADORES



Clase 3

Semántica operacional

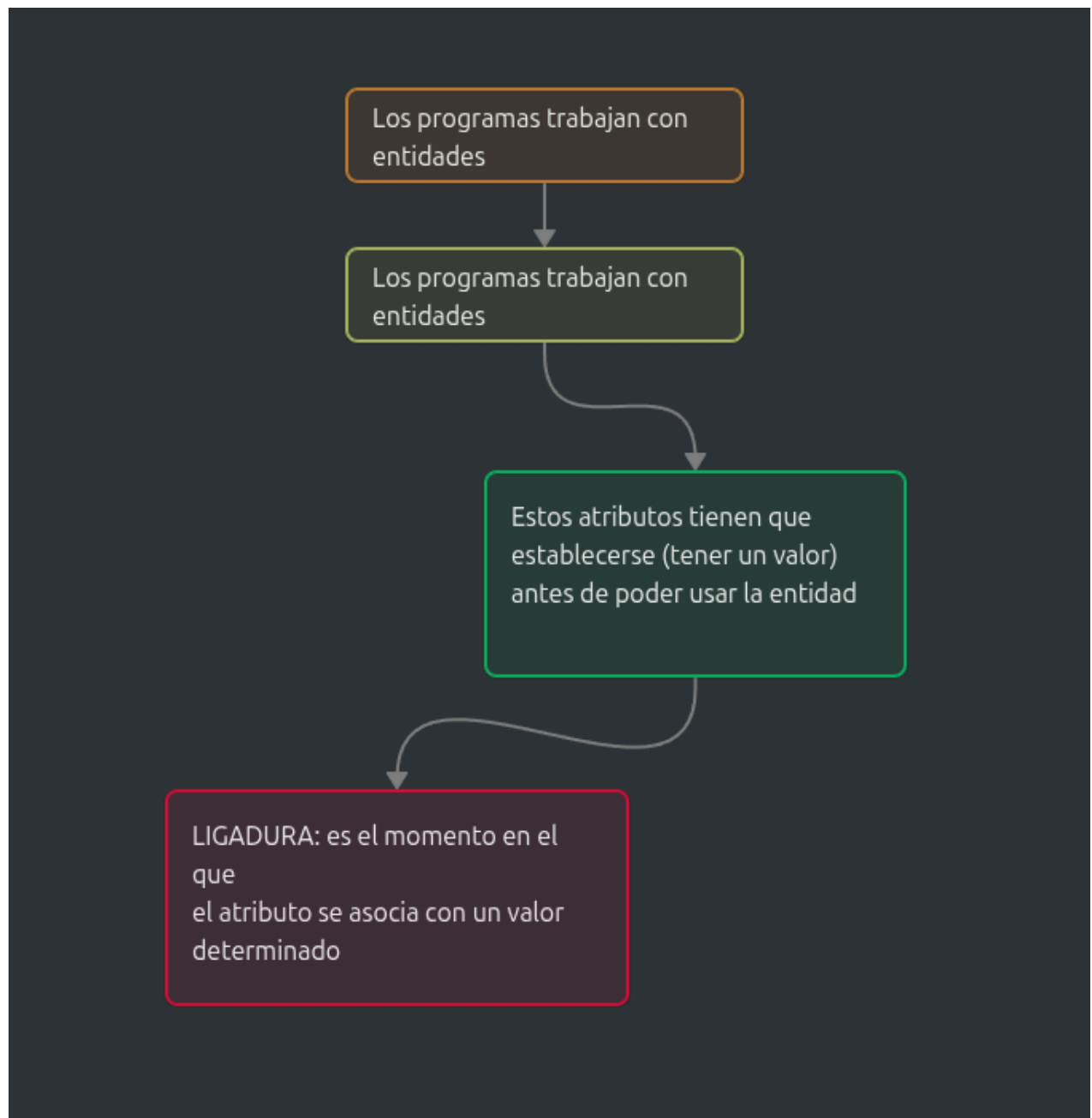
Permite describir el significado preciso de un programa y verificar el resultado final de la ejecución de un programa.

La semántica operacional es fundamental para diversos aspectos del proceso de desarrollo de software, como el diseño de lenguajes de programación, la verificación de programas y la comprensión de cómo se ejecutan los programas en un nivel más bajo.

Concepto de Ligadura (Binding)

Hay que asociar cada entidad a sus atributos.

Es un concepto central en la definición de la semántica de los lenguajes de programación.



Hay diferencias entre los lenguajes de programación en:

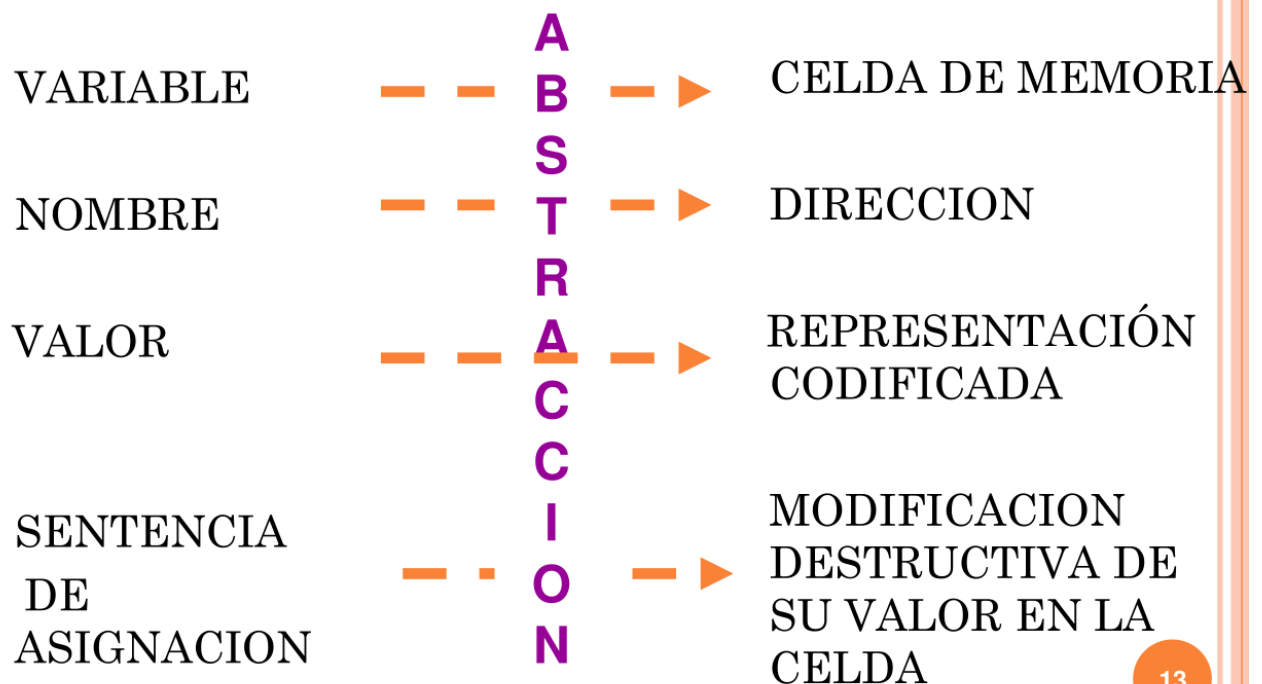
- El número de entidades
- El número de atributos que se les pueden ligar
- El momento de la ligadura cuando toma el valor (binding time)

Momento y estabilidad de la ligadura

Ligadura estática

1. Se establece antes de la ejecución.
 2. No se puede modificar.
- El termino estática referencia al binding time (1) y a su estabilidad
- Ligadura dinámica
1. Se establece durante la ejecución
 2. Si puede modificarse durante ejecución de acuerdo a alguna regla específica del lenguaje.
- Excepción: constantes (el binding es en ejecución/runtime pero no puede ser modificado luego de establecido)

Variables



13

Atributos de una variable

Es una 5-TUPLA:

1. Nombre: string de caracteres que se usa para referenciar a la variable. (identificador)
- Alcance: es el rango de instrucciones en el que se conoce el nombre, es visible, y puede ser referenciada

- Tipo: es el tipo de variables definidas, tiene asociadas rango de valores y conjunto de operaciones permitidas
- L-value: es el lugar de memoria asociado con la variable, está asociado al tiempo de vida (variables se alocan y desalocan)
- R-value: es el valor codificado almacenado en la ubicación de la variable

Aspectos del diseño del nombre que hay que conocer y validar

- El nombre es introducido por una sentencia de declaración
- Longitud máxima según lenguaje (se define en la etapa de definición del lenguaje)
- Caracteres aceptados en el nombre (conectores)
- Sensitivos a mayúsculas
- palabra reservada - palabra clave:
 - palabra clave: palabras propias del lenguaje
 - palabra reservada es aquella palabra clave que no puedo utilizar para asignar a un identificador, depende del lenguaje

Alcance

- Las instrucciones del programa pueden manipular las variables a través de su nombre dentro de su alcance.
- Afuera de ese alcance son invisibles
- Los diferentes lenguajes adoptan diferentes reglas para ligar el nombre de una variable a su alcance

1. Alcance Estático (Léxico)

- **Definición:** El alcance se define según la estructura léxica del programa. Permite ligar variables examinando el texto sin necesidad de ejecutarlo.
- **Funcionamiento:** Si una variable no está declarada en la unidad actual, se busca hacia afuera siguiendo la estructura de unidades contenidas (hacia las más externas). Si no se encuentra, reporta error.
- **Uso:** Es la regla adoptada por la mayoría de los lenguajes de programación.

2. Alcance Dinámico

- **Definición:** Define el alcance del nombre de la variable en términos de la **ejecución** del programa.
- **Funcionamiento:** Cada declaración extiende su efecto sobre todas las instrucciones ejecutadas posteriormente, hasta encontrar una nueva declaración con el mismo nombre.
- **Búsqueda:** Si una unidad referencia una variable no declarada en ella, la busca hacia afuera siguiendo la **secuencia de ejecución** (viendo quién llamó a la unidad que la necesita).
- Se usan cuando aparece referenciada en el código una variable que no es local, para saber a donde va a buscarla y saber a quien pertenece

Reglas de Alcance Estático	Reglas de Alcance Dinámico
Son las más utilizadas por los LP (C, PASCAL, ADA PYTHON, ETC.)	- Menos utilizadas por los LP - Más fáciles de implementar - Poco claras en cuanto a la programación y poco eficientes en la implementación

Clasificación de variable por alcance

- Global: Son todas las referencias a variables creadas en el Programa Principal.
- Local: Son todas las referencias a variables creadas dentro de una Unidad (programa o subprograma).
- No Local: Son todas las referencias a variables que se utilizan dentro de subprograma pero que no han sido creadas en la unidad. (son externas al subprograma)

Tipo

Se define el tipo de una variable cómo la especificación del:

1. conjunto de valores posibles que se pueden asociar a la variable y del
2. conjunto de operaciones permitidas sobre ellas(crear, acceder, modificar).

Ligar el tipo (conjunto de valores posibles y conjunto de operaciones asociadas) ayuda a:

- proteger a las variables de operaciones no permitidas
 - Realizar chequeo de tipos
 - Verificar el uso correcto de las variables (ej.. Cada lenguaje tiene sus reglas de combinaciones de tipos)
- Ayuda a que el compilador o intérprete detecte errores en forma temprana y a dar confiabilidad del código.

Clases de tipo

- Predefinidos - por el lenguaje
 - Son los tipos base que están descriptos en la Definición del Lenguaje (enteros, reales, flotantes, booleanos, etc....)
 - Cada uno tiene valores y operaciones asociadas
 - Los valores se ligan en la Implementación a representación de máquina según la arquitectura
- Definidos - por el usuario
 - Permiten al programador con la declaración de tipos definir nuevos tipos a partir de los tipos base y constructores predefinidos.
 - Esa declaración establece el binding en tiempo de traducción y heredan todas las operaciones de la representación de la estructura de datos base.
 - Permite al programador crear abstracciones, encapsular la lógica y datos, reutilizar código y mejorar la claridad y legibilidad del código.
 - Fundamentales para el desarrollo de programas complejos y para mantener un código organizado y mantenible.
 - Constructores: permiten crear otros tipos
 - TAD - Tipo Abstracto de Datos listas, colas, pilas, arboles, grafos, etc...
- Dependerá de cada lenguaje

Tipo de datos abstracto:

- No todos los lenguajes soportan la implementación de un “tipo” definido por el usuario llamado “Tipo de Datos Abstracto”
- Se debe asignar un nombre que lo identifique

- Se realiza la asociación del nuevo tipo con un set de operaciones que pueden ser usadas sobre sus instancias para manipular los objetos.
- Las operaciones son descritas como un set de rutinas que establece el programador y se especifican en la declaración del nuevo tipo.
- Cada TAD define un conjunto de operaciones permitidas (público), pero oculta los detalles de implementación interna (privado).
- No hay ligadura por defecto, el programador debe especificar la representación y las operaciones!
- TAD comunes: Listas, colas, pilas, arboles, grafos, etc...

Momentos de ligadura de Variable-tipo

1. Estático (static typing)

- **Definición:** La ligadura entre la variable y su tipo se especifica en la declaración; se realiza en **compilación** y no puede cambiarse en ejecución.
- **Protección y Detección:** Declarar variables de un tipo específico permite al compilador detectar automáticamente "violaciones de la semántica estática" (operaciones ilegales).
- **Chequeo de Tipos Estático:** Se realiza antes de la ejecución, lo que contribuye a la **detección temprana de errores** y mejora la **confiabilidad** del programa.
- Se realiza en forma:
 - Explícita: La ligadura se establece mediante una sentencia de declaración. La ventaja de las reside en la claridad de los programas y en una mayor confiabilidad
 - Implícita: Se deduce por "reglas propias del lenguaje".
 - Inferida: No se requiere que el TIPO de la variable sea declarado. El tipo se deduce/infiere automáticamente de los tipos de sus componentes. Se basa en el contexto del código y en el valor asignado a la variable de una asignación.

2. Dinámico (dynamic typing)

- en ejecución y puede modificarse.
- Tiene varios problemas:
 - Costo de implementación en TE
 - comprobación de tipos

- mantenimiento de tablas de Descriptores asociado a cada variable en el que se almacena el tipo actual
- cambio en el tamaño de la memoria asociada a la variable
- Chequeo dinámico
- Menor legibilidad y más posibilidad de errores

L-Value

Definición

Es la dirección del área de memoria ligada a la variable durante la ejecución.

Las instrucciones de un programa acceden a la variable por su L-Valor.

Tiempo de vida (lifetime) o extensión:

Es el Periodo de tiempo en que existe la ligadura

El tiempo de vida está muy ligado al L-Valor

- Es el tiempo en que está alocada la variable en memoria y en el cual el binding existe.
- Es desde que se solicita hasta que se libera

Alocación de memoria

Momento en que se reserva la memoria para una variable

Tipos de Momentos de Alocación de memoria:

- Estático
 - se hace en compilación (antes de la ejecución) cuando se carga el programa en memoria en zona de datos y perdura hasta fin de la ejecución
- Dinámico
 - se hace en tiempo de ejecución.

- Automática: cuando aparece una declaración de una variable en la ejecución
 - Explícita: requerida por el programador con una sentencia de creación, a través de algún constructor
- Persistente
 - Los objetos persistentes que existen en el entorno en el cual un programa es ejecutado, su tiempo de vida no tiene relación con el tiempo de ejecución del programa. Persisten más allá de la memoria.
El tipo dependerá del lenguaje

R-value

Definición

Es el valor codificado almacenado en la ubicación asociada a la variable (l-valor)

La codificación se interpreta de acuerdo con el tipo de la variable.
 interpreta un nro. entero si la variable es tipo int;
 interpreta una cadena si la variable es tipo char;

Momentos de ligadura - variable a valor

- Binding Dinámico de una variable a su valor e
 - el valor (r-valor) puede cambiar durante la ejecución con una asignación.
 - Constante: el valor (r-valor) no puede cambiar si se define como constante simbólica definida por el usuario
El binding dinámico varía según los lenguajes.
- Común en lenguajes imperativos

Momentos de ligadura - constante a valor

Binding Time de Expresiones (varía según los lenguajes)

- **En Pascal (Tiempo de Compilación):** El valor de una expresión constante se liga en compilación. El compilador reemplaza el nombre por el valor directamente, lo que hace al programa **más eficiente**.
- **En C y ADA (Tiempo de Ejecución):** Permiten expresiones que involucren variables. La ligadura ocurre recién en ejecución, cuando se crea la variable. Esto ofrece **mayor flexibilidad** pero impide ciertas optimizaciones previas.

Inicialización de una variable

Los lenguajes y sus versiones implementan diversas estrategias de inicialización:

1. Inicialización por defecto:
 - 1. Enteros se inicializan en 0
 - 2. Caracteres en blanco
 - 3. Funciones en VOID, etc.
2. Inicialización en la declaración
3. Estrategia Ignorar el problema:
 - 1. Toma como valor inicial lo que hay en memoria (la cadena de bits asociados al área de almacenamiento)
 - 2. Puede llevar a errores y requiere chequeos adicionales

Consideraciones adicionales

Variables anónimas (sin nombre) y referencias - Punteros

- **Acceso:** Algunos lenguajes permiten acceder a variables sin nombre a través del **r-valor** (contenido) de otra variable, denominado **referencia o puntero**.
- **Niveles de Acceso (Access Path):** * La referencia puede apuntar directamente al r-valor de una variable nombrada (**access path 0**).
 - Puede ser una cadena de referencias de longitud arbitraria (punteros a punteros).
- **Mecánica:** El r-valor de la variable puntero es la dirección (**l-valor**) de la otra variable.

Puntero

variable que sirve para señalar la posición de la memoria en la que se encuentra otro dato almacenando como valor, con la dirección de ese dato.

Variables anónimas (sin nombre) y referencias - Punteros y Alias

Alias

Se da si hay variables comparten un objeto en el mismo entorno de referencia. Sus caminos de acceso conducen al mismo objeto.

Por lo tanto el objeto compartido modificado por uno se modifica para todos los caminos

Ventajas

Compartir objetos se utiliza para mejorar la eficiencia.

Desventajas

Generan errores, el valor de una variable se puede modificar incluso cuando no se utiliza su nombre.

Generan programas que son difíciles de leer y de encontrar error.

Concepto de sobrecarga

Característica de algunos lenguajes de programación que permiten definir comportamientos personalizados para operadores, métodos, funciones.

Un mismo nombre que realiza algo distinto según el contexto.

Un nombre esta sobrecargado si en un momento referencia más de una entidad

- Debe estar permitido por el lenguaje.
- Debe haber suficiente información para permitir establecer la ligadura unívocamente. (por ejemplo determinarlo por su tipo)

Clase 4

Unidades de programas

Unidades

Los lenguajes de programación permiten que un programa esté compuesto de unidades.

Unidad → acción abstracta

En general se los llama rutinas (procedimientos o funciones)

Hay lenguajes que SOLO tienen “funciones” funciones y “simulan” simulan los procedimientos con funciones que devuelven "void" o “none”.

Nombre

String de caracteres que se usa para invocar a la rutina.

Alcance

rango de instrucciones donde se conoce su nombre.

- El alcance se extiende desde el punto de su declaración hasta algún constructor de cierre.
- Según el lenguaje puede ser estático o dinámico.
Activación: la llamada o invocación puede estar solo dentro del alcance de la rutina

Definición vs declaración

La **declaración**, declaración encabezado o prototipo es la firma de una función (nombre, tipo de retorno y parámetros) que informa al compilador sobre su existencia y estructura antes de ser utilizada.

Los prototipos son fundamentales para la organización del código, permitiendo el uso de funciones en múltiples archivos si se colocan en archivos de cabecera. (por ej. en archivos .h)

La **definición** es la declaración más el cuerpo. Proporciona el encabezado y el cuerpo real con las instrucciones que se ejecutarán cuando sea llamada.

Si el lenguaje distinguen entre la declaración y la definición de una rutina permite manejar esquemas de rutinas mutuamente recursivas.

Tipo

El encabezado de la rutina define el tipo de los parámetros y el tipo del valor de retorno (si lo hay).

Signatura: permite especificar el tipo de una rutina.

Un llamado a una rutina es correcto si está de acuerdo al tipo o signatura de la rutina.

La conformidad requiere la correspondencia de tipos entre parámetros formales y reales

L-value y R-value

- l-value: Es el lugar de memoria en el que se almacena el cuerpo p de la rutina.
- r-value: La llamada a la rutina causa la ejecución su código, eso constituye su r-valor.
 - estático: el caso más usual.
 - dinámica: variables de tipo rutina.
 - Se implementan a través de punteros a rutinas

Comunicación entre rutinas

- Ambiente no local
- Parámetros
 - Mayor legibilidad y modificabilidad
 - Parámetros formales: los que aparecen en la definición de la rutina
 - Parámetro reales: los que aparecen en la invocación de la rutina. (dato o rutina)

Ligadura entre parámetros reales y formales

- **Método posicional:** se ligan uno a uno
 - Variante: combinación con valores por defecto
- **Método por nombre:** Se ligan por el nombre, deben conocerse los nombres de los formales

Representación en ejecución

- La definición de la rutina especifica un proceso de cómputo.
- Cuando se invoca una rutina se ejecuta una instancia del proceso con los particulares valores de los parámetros.
- Instancia de la unidad: es la representación de la rutina en ejecución.

Procesador abstracto - SimplexEM

Utilidad

Nos servirá para comprender qué efecto causan las instrucciones del lenguaje al ser ejecutadas.

Se describe la semántica del lenguaje de programación a través de reglas de cada constructor del lenguaje traduciéndolo en una secuencia de instrucciones equivalentes del procesador abstracto

Procesador abstracto - SimplexEM

- Memoria de código: $C(y)$, donde y es una dirección. Dirección de la memoria de código,
- Memoria de datos: $D(y)$, donde y es una dirección o una celda de memoria.
- IP: Puntero a la instrucción que se está ejecutando. Luego de que se ejecuta una instrucción se aumenta y se pasa a la siguiente.

Instrucciones

Instrucciones

- SET: Asignación valores en la memoria de datos. Ej.: set tercer, source.
→ Copia el valor representado en source en la dirección representada por target.
- E/S: read y write permiten la comunicación con el exterior
- JUMP: bifurcación incondicional. Ej.: jump 47 → La próxima instrucción a ejecutarse será la que este almacenada en la dirección 47 de C.
- JUMPT: bifurcación condicional, bifurca si la expresión se evalúa como verdadera. Ej.: jumpt 47, $D[13] > D[8]$
- **Puntos de retorno:** Información que se debe guardar cuando una unidad es llamada. Contiene la siguiente dirección a ejecutar después

de que termine la subrutina

Ambiente de referencia

- Ambiente local: Variables locales, están almacenadas en su registro de activación.
- Ambiente no local: Variables no locales, están en los registros de activación de otras unidades.

Estructura de ejecución de los lenguajes de programación

- Estático: espacio fijo.
 - El espacio necesario para la ejecución se deduce del código
 - Todos los requerimientos de memoria necesarios se conocen antes de la ejecución
 - La alocaión puede hacerse estáticamente. Las unidades en la memoria perduran durante toda la ejecución del programa.
 - No puede haber recursión
- Basado en pilas (pascal, java, c): espacio predecible.
 - El espacio se deduce del código.
 - La memoria a utilizarse es predecible y sigue una disciplina last-in-first-out.
 - Las variables se alocan automáticamente y se desalocan cuando el alcance se termina
- Dinámico (Python, Ruby): espacio impredecible.
 - Todo se maneja de forma dinámica. Los datos se alocan en la zona de memoria heap.

Clase 5

Unidades de programa

Repaso Atributos

- **Nombre:** Declaración e invocación.
- **Alcance:** Dónde se conoce el identificador. Ambiente de referencia (global, local, no local).
- **Tipo:** Dado por los parámetros y valor de retorno (**Signatura**).
- **L-Valor:** Relacionado a dónde se almacenan las sentencias en la memoria.

- **R-Valor:** Relacionado al momento en que se hace la referencia a esa Rutina.
-

Procesador abstracto - Memoria

Concepto de Unidades

- Cuando se invoca una rutina se ejecuta una instancia del proceso con sus datos e información particular.
- La instancia de la unidad es la representación de la rutina en ejecución. Se tiene un registro de activación de la misma.
- El **ip** es el puntero a la instrucción siguiente a ejecutar en el código.

Zona de código (C)

- Aquí residen las sentencias.
- Se puede utilizar un lenguaje de un procesador abstracto como **Simple SEM**.
- **Instrucciones clave:**
 - **set** : Para asignar valores.
 - **jump** : Bifurcación incondicional.
 - **jump****t** : Bifurcación condicional.

Zona de datos (D)

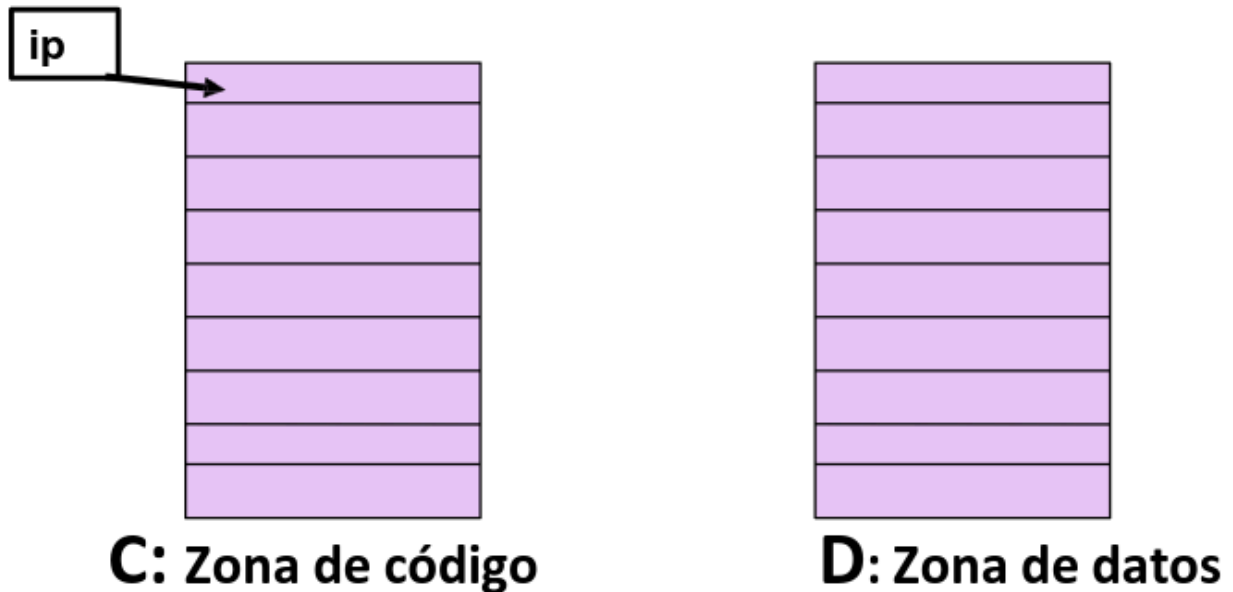
- Contiene toda la información que maneja la unidad y que debe registrarse.
 - Se utiliza una estructura llamada **Registro de Activación (RA)**.
-

El Registro de Activación (RA)

Es una estructura de datos en la memoria que se crea cada vez que se llama a una función o procedimiento. Contiene:

- Parámetros de entrada.

- Variables locales.
- Punto de retorno al código que la llamó.
- Enlaces (cadenas estática y dinámica).
- Valor de retorno.



Esquemas de Ejecución de las Unidades

Existen tres modelos principales para gestionar cómo se ejecutan las unidades y cómo se asigna su memoria:

1. Esquema Estático (Casos C1 y C2)

Se aplica en lenguajes donde no existe recursión ni anidamiento de rutinas.

Caso C1: Programa Simple

- Programa sin bloques internos ni subprogramas (solo el `main`).
- Toda la memoria se reserva de forma estática antes de comenzar.

Caso C2: Llamada y Retorno

- Permite subprogramas pero **sin anidamiento** ni **recursión**.
- **Características:**
 - Todo cargado en memoria desde el inicio.

- Existe **un solo RA por unidad** de programa.
- El RA contiene como mínimo: datos locales y el punto de retorno.
- Las variables tienen un **tiempo de vida estático** (existen siempre).

2. Esquema Basado en Pila (Casos C3 a C5)

Necesario cuando una unidad puede tener múltiples instancias activas simultáneamente (como en la recursión).

Caso C3: Recursión y Valor de Retorno

- Permite que las rutinas se llamen a sí mismas y devuelvan valores.
- **Funcionamiento:**
 - El tamaño del RA es fijo y conocido en compilación.
 - La cantidad de instancias es desconocida hasta la ejecución.
- **Ligaduras:**
 - **Variables:** El compilador las liga con su **desplazamiento (offset)** dentro del RA de forma estática.
 - **Dirección RA:** Se aloca dinámicamente en la pila en ejecución. La ligadura final con direcciones absolutas es dinámica.

Datos necesarios en el RA (Caso C3)

1. **Valor de retorno:** Espacio para el resultado si es una función.
2. **Punto de retorno:** Dirección de la instrucción a la que se debe volver.
3. **Enlace Dinámico (Dynamic Link):** Puntero a la base del RA de la unidad llamante (permite reconstruir la cadena de llamadas).

Gestión de la Pila

- **Current:** Puntero a la dirección base del RA que se está ejecutando actualmente.
- **Free:** Puntero a la próxima dirección libre en la pila.

Ejemplo: Semántica del CALL - RETURN

- Al llamar (**call**):

1. Aloca espacio para el valor de retorno.
 2. Salva el punto de retorno al código.
 3. Salva el enlace dinámico (el valor actual de **Current**).
 4. Actualiza **Current** al nuevo RA.
 5. Actualiza **Free** sumando el tamaño del nuevo RA.
 6. Transfiere el control a la unidad llamada.
- **Al retornar (**return**):**
 1. Reestablece el puntero **Free** usando **Current** .
 2. Reestablece **Current** usando el enlace dinámico guardado.
 3. Bifurca al punto de retorno.

Caso C4: Estructura de Bloque

Introduce la capacidad de tener bloques de código o rutinas dentro de otras.

C4': Bloques mediante Sentencias Compuestas

- Declaraciones locales dentro de bloques **{ ... }** .
- Las variables internas **enmascaran** a las externas si comparten nombre.
- Implementación: Puede ser estática (dentro del RA) o dinámica (al entrar al bloque).

C4'': Rutinas Anidadas (Ambiente No-Local)

Permite definir rutinas dentro de otras. El acceso a variables se resuelve mediante:

- **Cadena Dinámica:** Secuencia de llamadas (quién llamó a quién). Útil para desapilar.
- **Cadena Estática (Static Link):** Puntero al RA de la unidad que contiene estáticamente a la rutina actual. Permite el **Alcance Estático**.

Caso C5: Datos más Dinámicos

C5': Datos Semidinámicos (Arreglos Dinámicos)

- Variables cuyo tamaño se conoce al **activar la unidad** (ej: `array(1..N)`).
- Se usa un **Descriptor de Arreglo** en el RA que apunta al espacio alocado en la pila.

C5": Datos Dinámicos (Punteros y Heap)

- Alocados explícitamente (`new` , `malloc`) en el **Heap (Montículo)**.
- Su tiempo de vida no depende de la sentencia de asignación; viven mientras estén apuntados.

3. Esquema Dinámico (Caso C6)

- Típico de lenguajes como Python o JS.
- Tipado dinámico y reglas de alcance que se resuelven mayormente en ejecución.
- Se podrían tener reglas de tipado dinámicas y de alcance estático, pero en la práctica las propiedades dinámicas se adoptan juntas.
- Una propiedad dinámica significa que las ligaduras correspondientes se llevan a cabo en ejecución y no en compilación

Ejemplos Detallados

Ejemplo: Cadena Estática vs Dinámica

```
program Principal;  
  var x, y, z: integer;  
  
  procedure Uno;  
    var y: integer;  
    procedure Dos;  
      var m: integer;  
      begin  
        m := x + y;  // Acceso no-local  
      end;  
    begin  
      y := 4;
```

```

    Dos;
end;

procedure Tres;
begin
    x := 10;
    Uno;
end;

begin
    x := 1; y := 2; z := 3;
    Tres;
end.

```

Secuencia de llamadas: Principal -> Tres -> Uno -> Dos

Estado de la Pila en la ejecución de Dos :

1. **RA Principal:** (x=1, y=2, z=3).
2. **RA Tres:** Link Dinámico -> Principal. Link Estático -> Principal.
3. **RA Uno:** Link Dinámico -> Tres. Link Estático -> Principal.
4. **RA Dos:** Link Dinámico -> Uno. Link Estático -> Uno.

Resolución de $m := x + y$ en Dos :

- **Para y :** Dos mira su Link Estático (apunta a **Uno**). Encuentra la **y** local de Uno (**y=4**).
- **Para x :** Dos mira su Link Estático (**Uno**). Como Uno no tiene **x** , Dos sigue el Link Estático de Uno (apunta a **Principal**). Encuentra la **x** de Principal (**x=10** , modificada previamente por **Tres**).

🔗 Conclusión

La **Cadena Dinámica** sirve para saber a dónde volver (desapilar). La **Cadena Estática** sirve para encontrar variables en rutinas contenedoras (alcance).

Resumen de Variables según el Tiempo de Vida

Caso	Tipo de Variable	Esquema	Ubicación
C1 - C2	Estáticas	Estático	Zona de Datos fija
C3 - C5	Semiestáticas / Automáticas	Basado en Pila	Pila (Stack)
C6	Dinámicas	Dinámico	Montículo (Heap)

Clase 6

Clase 6: Semántica Operacional – Comunicación entre Rutinas y Pasaje de Parámetros

Contexto General

En el diseño de lenguajes de programación, la modularización a través de rutinas exige mecanismos robustos para el intercambio de datos. Esta clase profundiza en la semántica operacional de la comunicación, analizando desde el acceso a ambientes no locales hasta los diversos modos de pasaje de parámetros y sus implicancias en la seguridad y eficiencia del software.

I. Abstracción de Rutinas y Subprogramas

Las rutinas (también denominadas subprogramas, procedimientos o funciones según el lenguaje y la bibliografía) constituyen la unidad fundamental de abstracción. Permiten encapsular sentencias y definir nuevas operaciones que pueden ser invocadas mediante un protocolo de **call/return**.

Rutinas

Son construcciones que permiten al programador definir una acción abstracta, ocultando los detalles de su implementación. Su invocación representa el uso de dicha abstracción, donde el control se transfiere a la rutina y, al finalizar, retorna al punto inmediatamente posterior a la llamada.

Clasificación Semántica

- **Procedimientos:** Construcciones que no devuelven un valor de forma directa. Su impacto se observa a través de efectos colaterales, como la modificación de variables no locales o parámetros pasados por referencia.
- **Funciones:** Semánticamente similares a las funciones matemáticas. Se invocan dentro de expresiones y devuelven un valor explícito como resultado. Aunque idealmente deberían ser puras, en muchos lenguajes imperativos pueden producir efectos colaterales.

🔥 Important

El ocultamiento de la implementación es clave: el programador no necesita conocer *cómo* está programada la rutina para utilizarla, solo su interfaz de comunicación.

Síntesis de la sección

La abstracción mediante rutinas fomenta la modularidad y la reutilización, pero introduce el problema central de la clase: **¿Cómo intercambian información estas unidades de forma segura y eficiente?**

II. Mecanismos de Comunicación de Datos

Existen tres vías principales para que las rutinas compartan información, cada una con distintos niveles de claridad y riesgo.

1. Variables Locales y No Locales

- **Variables Locales:** Protegidas dentro de la rutina, invisibles al exterior.
- **Ambiente No Local Implícito:** Basado en reglas de alcance (Dinámico o Estático). El lenguaje busca identificadores fuera de la rutina actual. Es un mecanismo propenso a errores ("poco claro") ya que depende de quién llamó a quién o de la estructura léxica.
- **Ambiente Común Explícito:** El programador define áreas de memoria compartida (ej. `COMMON` en FORTRAN, `packages` en ADA).

2. Pasaje de Parámetros

Representa la forma más clara y recomendada de comunicación. La dependencia se vuelve explícita en la firma de la rutina.

Parámetro Real vs. Formal

- **Parámetro Real (Argumento):** Es el valor, variable o expresión que se envía en el momento de la **invocación**.
- **Parámetro Formal:** Es la variable declarada en la **cabecera** de la rutina que actúa como receptor del dato.

Síntesis de la sección

Mientras que el acceso a variables no locales es implícito y riesgoso, el pasaje de parámetros es explícito, lo que mejora la legibilidad y facilita el mantenimiento al reducir el acoplamiento oculto.

III. Vinculación y Ligadura de Parámetros

La vinculación entre parámetros reales y formales ocurre en dos etapas críticas:

1. Evaluación

Se evalúan los parámetros reales (expresiones, variables) antes de transferir el control a la unidad llamada. Se debe verificar que el estado sea consistente antes del pasaje.

2. Ligadura (Binding)

Establece la correspondencia exacta entre los argumentos enviados y los parámetros declarados.

- **Por Posición:** El método más común; la correspondencia se da por el orden físico en la lista de parámetros.

- **Por Nombre (Keyword):** Se vincula mediante el nombre explícito del parámetro formal (ej. en Python o Ada: `area(base=10, altura=5)`). Permite cambiar el orden y mejora la documentación del código.
- **Valores por Defecto:** Permiten omitir argumentos en la llamada, reduciendo la cantidad de sobrecarga necesaria.

Important

La ligadura por nombre requiere que el programador conozca los nombres de los parámetros formales, lo que introduce una dependencia con la implementación interna de la rutina.

Síntesis de la sección

El diseño de la vinculación impacta en la flexibilidad del lenguaje. Los valores por defecto y las palabras clave reducen la verbosidad y los errores por posición incorrecta.

IV. Modos de Pasaje de Datos (Semántica de Entrada/Salida)

La clasificación de los parámetros según cómo fluye la información entre la rutina llamadora y la llamada es fundamental para entender el comportamiento del programa.

1. Modo IN (Entrada)

El flujo es unidireccional hacia la rutina llamada.

- **Por Valor:** Se realiza una copia física del parámetro real en el formal. Los cambios en la rutina no afectan al original.
 - *Ventaja:* Seguridad y aislamiento.
 - *Desventaja:* Costo de copia (tiempo y memoria) para estructuras grandes.
- **Por Valor Constante:** Similar al pasaje por valor, pero el lenguaje prohíbe explícitamente cualquier modificación al parámetro formal (ej.

`const` en C++).

2. Modo OUT (Salida)

El flujo es unidireccional hacia la rutina llamadora al finalizar la ejecución.

- **Por Resultado:** El parámetro formal no tiene valor inicial. Al terminar la rutina, su valor se copia de vuelta al parámetro real (que debe ser un L-valor).

3. Modo IN/OUT (Bidireccional)

- **Por Valor-Resultado:** Copia el valor al entrar y copia el resultado al salir. No hay vínculo durante la ejecución; la rutina trabaja con una copia local.
- **Por Referencia (Pasaje por Variable):** Se pasa la dirección de memoria (puntero) del parámetro real. La rutina opera directamente sobre la variable original.
 - *Ventaja:* Eficiencia extrema (sin copias).
 - *Riesgo:* **Aliasing**.
- **Por Nombre (Lazy Evaluation):** El parámetro formal se reemplaza textualmente por el real. La expresión se evalúa cada vez que se usa. Se implementa mediante **Thunks**.

🔗 El problema del Aliasing

Ocurre cuando dos nombres se refieren a la misma celda de memoria. Es común en el pasaje por referencia (ej. pasar una variable global como parámetro por referencia). Esto rompe la transparencia referencial y dificulta la depuración.

Síntesis de la sección

La elección del modo de pasaje es un trade-off entre **seguridad** (Modo IN/Valor) y **eficiencia/flexibilidad** (Modo IN-OUT/Referencia).

V. Subprogramas como Parámetros

Cuando una rutina recibe a otra como parámetro, surge el problema del **Ambiente de Referencia** para las variables no locales de la rutina pasada.

Reglas de Ligadura (Binding)

1. **Ligadura Profunda (Deep Binding):** El ambiente de referencia es aquel donde se **definió** el subprograma. Es el estándar para lenguajes con alcance estático.
2. **Ligadura Superficial (Shallow Binding):** El ambiente es aquel que **invoca** al subprograma. Típico de lenguajes con alcance dinámico.
3. **Ligadura Ad-hoc:** El ambiente es aquel donde el subprograma es **pasado** como parámetro.

Síntesis de la sección

La ligadura profunda es la más coherente con el alcance estático, asegurando que la función siempre acceda a las variables que "veía" cuando fue escrita, independientemente de dónde se ejecute.

Conceptos clave para apuntes

- **Abstracción:** Ocultamiento de detalles de implementación mediante rutinas.
- **L-valor:** Dirección de memoria necesaria para modos de pasaje como Referencia o Resultado.
- **Aliasing:** Situación de riesgo donde múltiples nombres acceden a la misma memoria.
- **Thunk:** Fragmento de código para implementar evaluación diferida en el pasaje por nombre.
- **Referencia vs. Valor:** Decisión crítica entre eficiencia de memoria y seguridad de datos.

Resumen integrador

La comunicación entre rutinas es el pilar de la modularidad. Mientras que los lenguajes antiguos dependían de ambientes compartidos, los lenguajes modernos priorizan el **pasaje de parámetros**. El programador debe discernir entre los diversos modos (Valor, Referencia, etc.) para balancear la integridad de los datos frente al rendimiento del sistema, siempre alerta a los efectos colaterales y al aliasing.

Clase 7

Clase 7: Tipos de Datos

La organización de los datos a través del concepto de **Tipo de Datos** es uno de los pilares fundamentales en el diseño de lenguajes de programación. No se trata solo de clasificar información, sino de dotar a los datos de un comportamiento semántico y sentido dentro del sistema.

Definición

Un **Tipo de Datos** se define como un conjunto de valores y un conjunto de operaciones que se pueden utilizar para manipular dichos valores.

I. Evolución Histórica y Concepto

En los inicios de la computación, los lenguajes tenían estructuras de datos muy limitadas, restringidas a los tipos básicos definidos por el hardware y el lenguaje mismo. Con el tiempo, surgió la necesidad de soportar una mayor variedad de aplicaciones, lo que impulsó la evolución hacia la **legibilidad** y **modificabilidad** del código.

- **Primeros Lenguajes:** Modelado limitado a tipos básicos (enteros, reales).
- **Tipos Definidos por el Usuario:** Permiten al programador especificar agrupaciones de datos elementales para resolver problemas específicos.
- **Tipos de Datos Abstractos (TAD):** Separan la representación interna de los datos del conjunto de operaciones disponibles, haciéndolas

invisibles al usuario.

- **Clases:** Representan la evolución final del TAD en el paradigma orientado a objetos.

II. Categorías de Tipos de Datos

Podemos clasificar los tipos de datos en dos grandes ejes según su origen y su estructura:

1. Predefinidos (Primitivos/Built-in)

Reflejan el comportamiento del hardware subyacente y son abstracciones del mismo (como el byte o la palabra de memoria).

- **Elementales:** Enteros, Reales, Caracteres, Booleanos.
- **Compuestos:** Strings (en muchos lenguajes).

Important

Las ventajas de los tipos predefinidos incluyen la **verificación estática**, el control de precisión, la desambiguación de operadores y la invisibilidad de la representación física.

2. Definidos por el Usuario

Creados por el programador mediante **constructores de tipos**. Permiten separar la especificación de la implementación.

- **Elementales:** Enumerados (conjunto finito de valores con nombre).
- **Compuestos:** Arreglos, Registros, Listas, Conjuntos.

III. Constructores de Tipos Compuestos

Para generar tipos complejos a partir de tipos más simples, los lenguajes proveen mecanismos denominados constructores:

1. Producto Cartesiano (Registros/Structs)

Define un conjunto de n-tuplas ordenadas. En lenguajes como C, se implementa mediante `struct`.

- **Estructura:** Permite agrupar campos de diferentes tipos bajo un mismo nombre.
- **Acceso:** Generalmente mediante notación puntual (`objeto.campo`).

2. Correspondencia Finita (Arreglos)

Funciones de un dominio de tipo **índice** (generalmente un subrango de enteros) a un rango de valores de un tipo determinado.

- **Ejemplo:** En C, `char digits[10]` mapea los índices `[0..9]` al tipo `char`.

3. Unión y Unión Discriminada

Permite que una misma ubicación de memoria almacene diferentes tipos de datos en distintos momentos de la ejecución (disyunción de tipos).

- **Unión (C):** No es segura; el programador debe recordar qué tipo guardó.
- **Unión Discriminada:** Agrega un **discriminante** (tag) para indicar qué variante está activa. El chequeo de tipos debe hacerse en ejecución.
- **Problemas:** El discriminante y las variantes pueden manejarse independientemente, lo que permite omitir chequeos.

4. Recursión

Un tipo recursivo se define como una estructura que puede contener componentes de su mismo tipo. Es la base para implementar estructuras de tamaño arbitrario y complejidad variable (listas ligadas, árboles).

IV. Punteros y Manejo de Memoria

Los punteros son referencias a objetos en memoria y son el mecanismo principal para implementar estructuras de datos de tamaño arbitrario.

Note

Un puntero es una variable cuyo r-valor es una referencia (dirección de memoria) a otro objeto.

Inseguridades de los Punteros

El uso de punteros es potente pero peligroso (comparable al `go to` de los datos):

1. **Violación de tipos:** Acceder a una posición de memoria interpretándola con un tipo incorrecto (ej. aritmética de punteros).
2. **Referencias sueltas (Dangling references):** Apuntar a una dirección que ya fue liberada.
3. **Punteros no inicializados:** Acceso descontrolado a memoria basura.
4. **Alias:** Múltiples punteros apuntando al mismo objeto, dificultando el rastreo de cambios.
5. **Objetos perdidos (Memory Leaks):** Memoria alocada que ya no es accesible (basura) pero no fue liberada.

V. Gestión de Memoria: Garbage Collection

Existen dos enfoques principales para liberar la memoria del **Heap**:

- **Explícita:** El programador decide cuándo liberar (ej. `free` en C, `dispose` en Pascal). Eficiente pero propensa a errores (dangling pointers, leaks).
- **Implícita (Garbage Collector):** El sistema detecta automáticamente qué objetos ya no son accesibles ("basura") y libera su espacio.

Estrategias de Recolección (GC)

1. **Cuenta de Referencias:** Mantiene un contador de punteros hacia el objeto. Libera cuando llega a cero. **Problema:** No gestiona estructuras

circulares.

2. **Recolección por Trazado (Mark and Sweep):** Marca objetos alcanzables desde la pila y elimina el resto.
3. **Stop and Copy:** Divide el Heap en dos mitades. Copia los objetos útiles de una mitad a la otra, compactándolos y eliminando la fragmentación externa.
4. **Recolección Generacional:** Divide el Heap según la "edad" de los objetos (Joven, Vieja). Asume que la mayoría de los objetos mueren jóvenes, reduciendo el costo de inspección.

Comparación de Implementaciones

Característica	Java	PHP	Python	Javascript
Mecanismo	Generacional	Cuenta de Referencias	Cuenta de Referencias	Marca Barrido (actual)
Reconocimiento	3 Generaciones (Joven, Vieja, Perm)	Contenedor z-val (is_ref, refcount)	Interfaz opcional (GC cíclico)	Definido por el motor (V8, etc)
Configuración	Serial, Parallel, CMS, G1	Corregido fugas en 5.3.0	Deshabilitable, ajuste freq.	Depende del navegador

VI. El Caso de Rust: Ownership

Rust introduce un tercer camino para la gestión de memoria que no depende de un Garbage Collector ni de la liberación manual tradicional.

Reglas de Propiedad (Ownership)

1. Cada valor tiene una variable que es su **propietario**.
2. Solo puede haber un propietario a la vez.
3. El propietario puede **prestar** o **entregar** la propiedad.
4. Cuando el propietario sale del alcance (scope), el valor se descarta automáticamente (**drop**).

Important

Rust no permite que dos punteros apunten al mismo lugar para evitar errores de doble liberación. Si se asigna una variable a otra, la representación se **mueve** (move), invalidando la variable original.

VII. Tipos de Datos Abstractos (TAD)

Un TAD es un tipo definido por el usuario que satisface el **encapsulamiento** y el **ocultamiento de la información**.

Especificación Formal

Se compone de:

- **Sintaxis:** Define la firma de las operaciones (Argumentos -> Resultado).
- **Semántica:** Define el comportamiento mediante axiomas (Expresiones de resultado).
- **Constructores:** Operaciones que inicializan el tipo.

Implementaciones por Lenguaje

- **Ada:** Paquete (**package**). Permite separar especificación (visible) de implementación (privada).
- **Modula-2:** Módulo.
- **Java/C++:** Clase. Java trata todos los objetos como referenciados por punteros de forma uniforme.

VIII. Sistema de Tipos

Conjunto de reglas para estructurar y organizar los tipos, con el fin de escribir **programas seguros**.

Tipado y Seguridad

- **Fuerte vs Débil:** Un sistema es **fuerte** si impone restricciones que aseguran la ausencia de errores de tipo en ejecución (**Type Safety**).
- **Estático vs Dinámico:** Momento de la ligadura (compilación vs ejecución). El tipado dinámico provoca más comprobaciones en runtime.

Reglas de Inferencia

El compilador puede deducir el tipo de una entidad sin declaración explícita basándose en:

- El operador utilizado (ej. `mod` implica enteros).
- Los operandos.
- Los argumentos de una función.
- El tipo del resultado esperado.

Reglas de Equivalencia y Conversión

1. **Equivalencia:** Por **Nombre** (mismo nombre declarado) o por **Estructura** (componentes idénticos).
2. **Compatibilidad:** Un tipo es compatible si es equivalente o se puede convertir.
3. **Conversión:**
 - **Coerción:** Automática (Widening: sin pérdida; Narrowing: posible pérdida).
 - **Casting:** Explícita, forzada por el programador.

IX. Polimorfismo

Permite que una entidad tome valores de diferentes tipos.

1. Polimorfismo Universal

- **Paramétrico:** Tipos con parámetros (ej. `List<T>`). Provee uniformidad estructural.

- **Inclusión (Herencia):** Modelado de subtipos. Un subtipo T' es un subconjunto de los valores de T con las mismas operaciones.

2. Polimorfismo Ad-hoc

- **Sobrecarga:** Un mismo identificador ligado a distintas abstracciones.
 - **Coerción:** El operador se aplica de forma segura sobre un tipo diferente al esperado.
-

Síntesis de la clase

- Los tipos de datos abstraen **valores y operaciones**, evolucionando hacia el ocultamiento de la representación física.
- La gestión de memoria en el Heap varía desde la liberación manual (C) hasta el **Garbage Collector** (Java, Python) y el modelo de **Ownership** (Rust).
- Las estrategias de GC como **Stop and Copy** y **Generacional** optimizan el rendimiento y evitan la fragmentación.
- Un **TAD** robusto requiere una especificación formal clara (sintaxis y semántica) para garantizar la modularidad.
- El **Sistema de Tipos** balancea **Seguridad vs Flexibilidad**, utilizando inferencia y polimorfismo para mejorar la expresividad del código.

Conceptos clave para apuntes

- **Ownership:** Gestión de memoria basada en pertenencia y alcance (Rust).
- **Dangling Pointer:** Puntero a memoria ya liberada.
- **Inferencia de tipos:** Deducción automática del tipo por el contexto.
- **Unión Discriminada:** Alternativa segura a la unión simple mediante tags.
- **Stop and World:** Pausa en la ejecución durante la recolección de basura.

Clase 9: Estructuras de Control

I. Contexto General

Las **estructuras de control** son el medio fundamental por el cual los programadores especifican el flujo de ejecución entre los componentes de un programa. En los lenguajes convencionales, estas estructuras permiten organizar el código en relación con el flujo de control, dividiéndose principalmente en dos niveles: **Nivel de Unidad** (pasaje de parámetros, excepciones, call-return) y **Nivel de Sentencia**.

Note

Esta clase se centra en el **Nivel de Sentencia**, el cual se subdivide en tres grupos fundamentales: **Secuencia**, **Selección** e **Iteración**.

II. Conceptos Clave

Definición

Secuencia: Flujo de control más simple donde se ejecutan instrucciones en un orden específico (de arriba hacia abajo) sin desviaciones.

Selección (o decisiones): Estructuras que permiten tomar decisiones basadas en condiciones, dirigiendo el flujo hacia diferentes caminos según el resultado de la evaluación.

Iteración (o bucles): Mecanismos que permiten repetir un bloque de código múltiples veces hasta que se cumpla una condición de salida.

III. Desarrollo por Subtema

1. Secuencia y Sentencias de Asignación

La secuencia se basa en la ejecución de una sentencia a continuación de otra. El delimitador más común es el `;`, aunque existen lenguajes orientados a línea (Fortran, Basic, Python) o que permiten bloques compuestos (`begin-end` en Pascal/Ada, `{}` en C/Java).

Distinción entre Asignación y Expresión

Es crucial diferenciar entre una **sentencia de asignación** y una **expresión**:

- **Asignación:** Devuelve el *r-valor* de una expresión y lo asigna al *l-valor*, modificando el estado de la memoria.
- **Expresión:** Devuelve un valor obtenido al combinar operandos a través de operadores, sin necesariamente modificar la memoria.

🔥 Important

En lenguajes como C, la asignación se define como una "expresión con efectos colaterales", lo que permite asignaciones múltiples (`a=b=c=0`) o asignaciones dentro de condiciones (`if (i=30)`).

2. Sentencia de Selección (IF)

La semántica del `IF` es similar entre lenguajes (ejecución condicional), pero su sintaxis ha evolucionado para resolver problemas de **ambigüedad** (especialmente en `if-then-else` anidados).

Problema de la Ambigüedad (Dangling Else)

En lenguajes como Algol 60, no estaba claro a qué `if` pertenecía un `else` en estructuras anidadas. Para resolverlo, se aplican diferentes reglas:

1. **Regla de Proximidad:** El `else` se empareja con el `if` más cercano (C, Pascal).
2. **Cierres Explícitos:** Uso de palabras clave de cierre como `fi` (Algol 68), `end if` (Ada) o `end` (Modula-2).
3. **Bloques Delimitados:** Obligatoriedad de usar `{}` o `begin-end`.

Selección en Python

Python introduce una sintaxis que elimina la ambigüedad mediante la **indentación obligatoria** y la palabra clave `elif`.

3. Evaluación de Expresiones: Circuito Corto

Se refiere a la técnica de evaluación de expresiones booleanas (`AND` , `OR`) donde no se evalúan todos los operandos si el resultado ya está determinado.

Definición

Circuito Corto (Lazy Evaluation): Los operandos se evalúan de izquierda a derecha solo hasta encontrar el primero que determine el resultado final.

Ventajas del Circuito Corto	Descripción
Evita Errores	Previene divisiones por cero o acceso a punteros nulos (<code>x != 0 && 10/x</code>).
Optimización	Ahorra tiempo al no ejecutar funciones o chequeos costosos innecesariamente.

4. Selección Múltiple (Switch/Case)

Permite elegir entre múltiples caminos sin recurrir a `if` anidados.

- **Pascal:** Usa `case-of-else-end`. Inseguro si no se cubre el `else`.
- **Ada:** Reglas estrictas, debe contemplar todos los valores posibles (o usar `others`).
- **C/C++ (Switch):** Utiliza etiquetas constantes. Presenta el fenómeno de **Falling Through** (si se omite el `break`, la ejecución continúa en el siguiente caso).
- **Ruby:** Muy flexible, permite evaluar cualquier valor (cadenas, rangos).

IV. Iteración (Bucles)

La iteración permite ejecutar un cuerpo de bucle repetidamente.

1. Bucle FOR (Contadores)

- **Fortran (DO):** Originalmente se ejecutaba siempre al menos una vez; versiones modernas evalúan al inicio.
- **Pascal/Ada:** La variable de control es ordinal. En Ada, no necesita declaración previa y desaparece al salir del bucle.
- **C/C++:** Muy flexible, consta de 3 partes (inicialización, test, incremento). Permite bucles infinitos `for(;;)` .
- **Python:** Itera sobre secuencias (listas, tuplas) o rangos numéricos usando `range()` .

2. Bucle WHILE (Condicionales)

- **Chequeo al inicio:** `while` (Pascal, C, Ada, Python). El cuerpo puede no ejecutarse nunca.
- **Chequeo al final:** `repeat-until` (Pascal) o `do-while` (C). Se ejecuta al menos una vez.

V. Síntesis Final

Las estructuras de control son la columna vertebral de la lógica de programación. Comprender sus diferencias sintácticas y semánticas entre lenguajes (como el manejo de la ambigüedad en los `if` o el "falling through" en los `switch`) es crucial para escribir código seguro, legible y eficiente.

Conceptos clave para apuntes

- ☐ Diferencia entre **Nivel de Unidad** y **Nivel de Sentencia**.
- ☐ La importancia del **l-valor** en las asignaciones.
- ☐ Resolución de la ambigüedad en condicionales anidados.
- ☐ Mecanismo y beneficios de la **evaluación de circuito corto**.
- ☐ Diferencias entre bucles con chequeo al inicio vs. chequeo al final.

